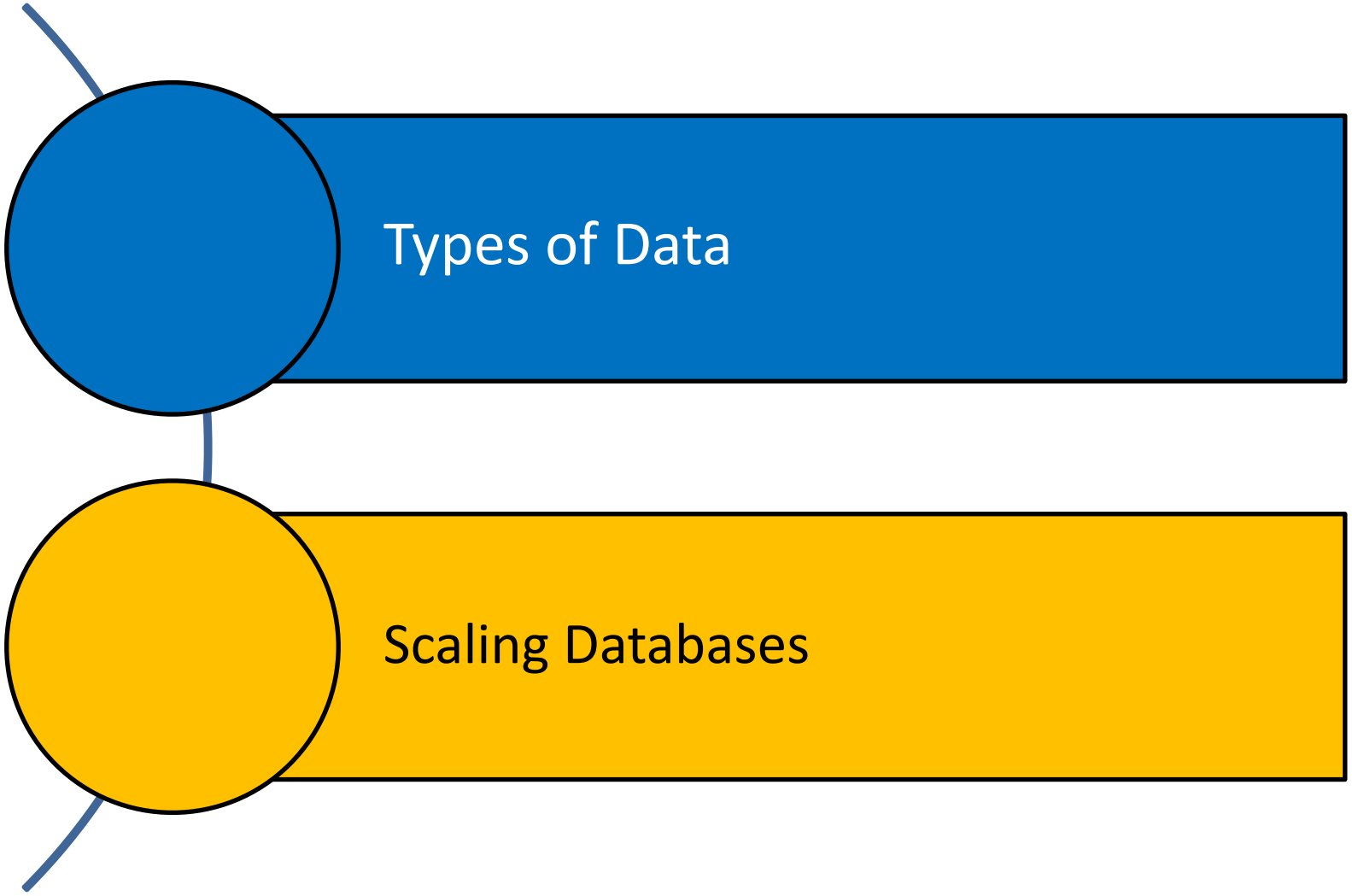
# Part I- Databases

Prof. Aparna Kulkarni

# Outline



# Types of Data

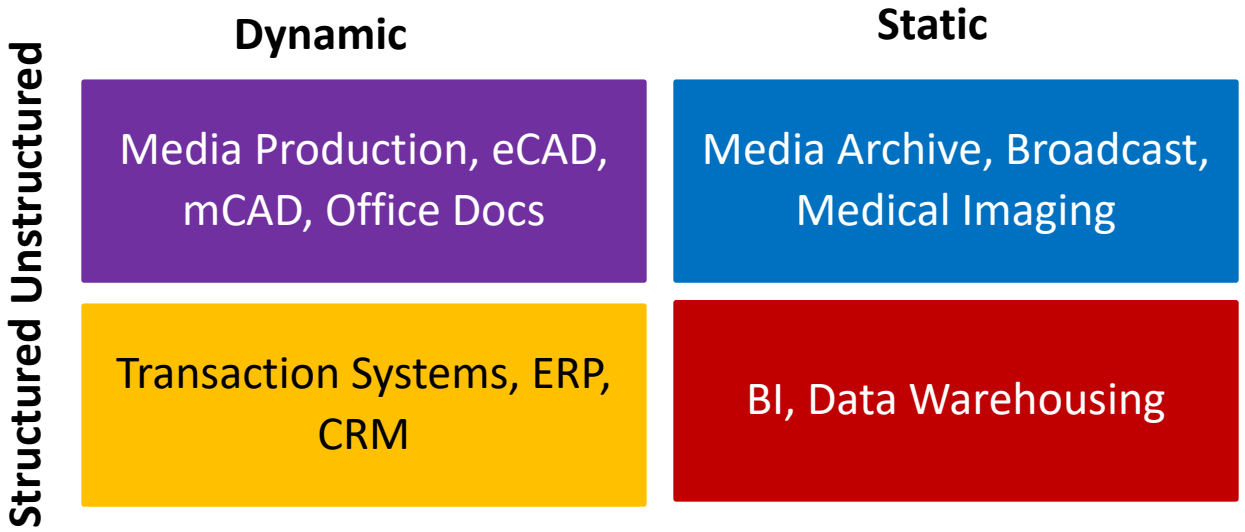
- Data can be broadly classified into four types:
  1. Structured Data:
     - Have a predefined model, which organizes data into a form that is relatively easy to store, process, retrieve and manage
     - E.g., relational data
  2. Unstructured Data:
     - Opposite of structured data
     - E.g., Flat binary files containing text, video or audio
     - Note: data is not completely devoid of a structure (e.g., an audio file may still have an encoding structure and some metadata associated with it)

# Types of Data

- Data can be broadly classified into four types:
  3. Dynamic Data:
     - Data that changes relatively frequently
     - E.g., office documents and transactional entries in a financial database
  4. Static Data:
     - Opposite of dynamic data
     - E.g., Medical imaging data from MRI or CT scans

# Why Classifying Data?

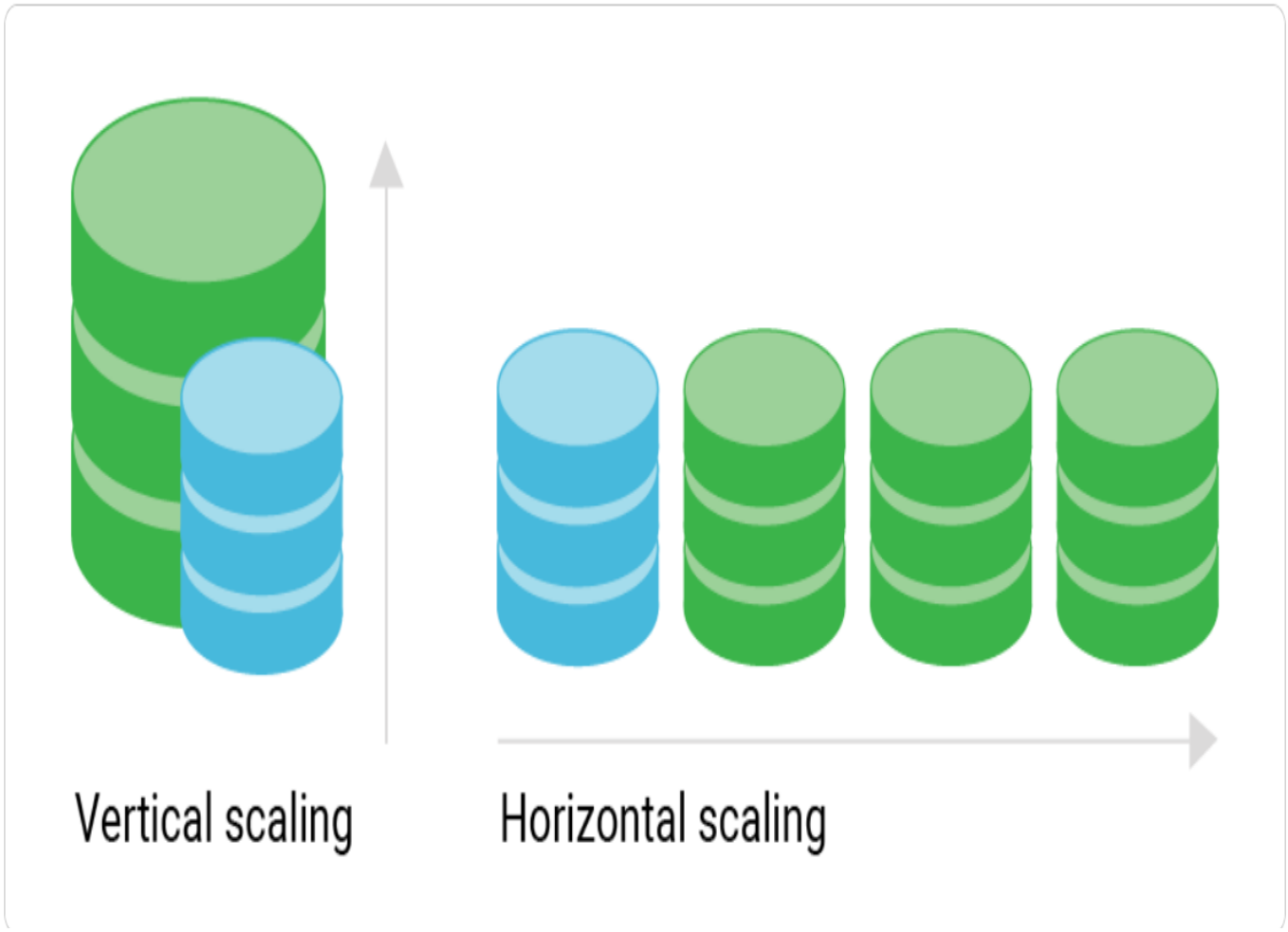
- Segmenting data into one of the following 4 quadrants can help in designing and developing a pertaining storage solution




- Relational databases are usually used for structured data

- File systems or *NoSQL databases* can be used for (static), unstructured data (*more on these later*)

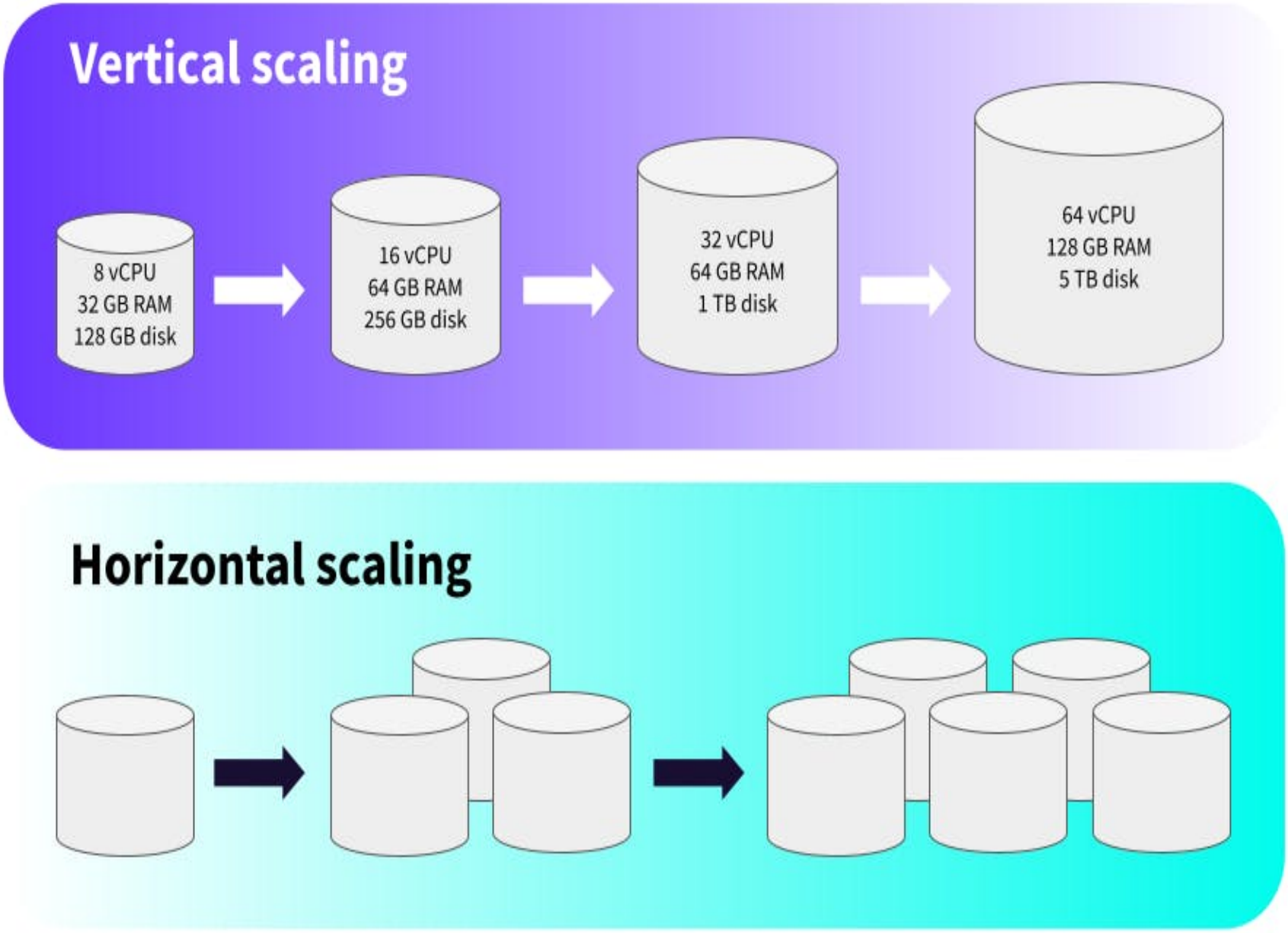
Scaling Databases


Vertical scaling
Horizontal scaling

# Horizontal Scaling

- refers to adding additional nodes or machines to your infrastructure to cope with new demands.

- If you are hosting an application on a server and find that it no longer has the capacity or capabilities to handle traffic, adding a server may be your solution.

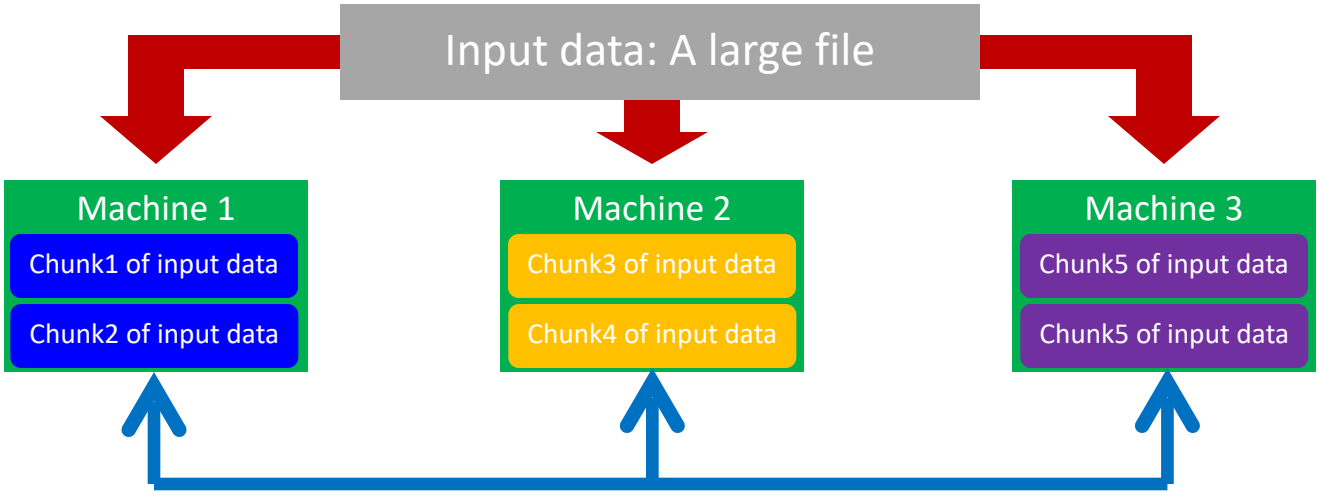

Vertical scaling
8 vCPU
32 GB RAM
128 GB disk
16 vCPU
64 GB RAM
256 GB disk
32 vCPU
64 GB RAM
1 TB disk
64 vCPU
128 GB RAM
5 TB disk
Horizontal scaling

# Scaling Traditional Databases

- Traditional RDBMSs can be either scaled:
  - Vertically (or Up)
    - Can be achieved by hardware upgrades (e.g., faster CPU, more memory, or larger disk)
    - Limited by the amount of CPU, RAM and disk that can be configured on a single machine
  - Horizontally (or Out)
    - Can be achieved by adding more machines
    - Requires database *sharding* and probably *replication*
    - Limited by the Read-to-Write ratio and communication overhead

# Why Sharding Data?

- Data is typically *sharded* (or *striped*) to allow for concurrent/parallel accesses



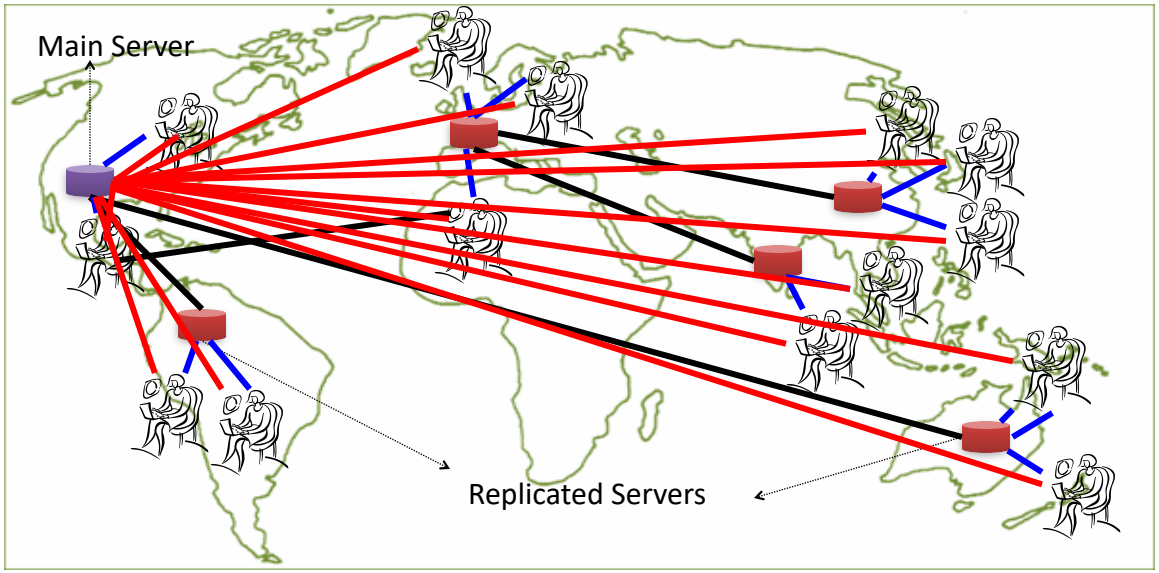

E.g., Chunks 1, 3 and 5 can be accessed in parallel

# Some Guidelines

- Here are some guidelines to effectively benefit from parallelization:
  1. Maximize the fraction of your program that can be parallelized
  2. Balance the workload of parallel processes
  3. Minimize the time spent for communication

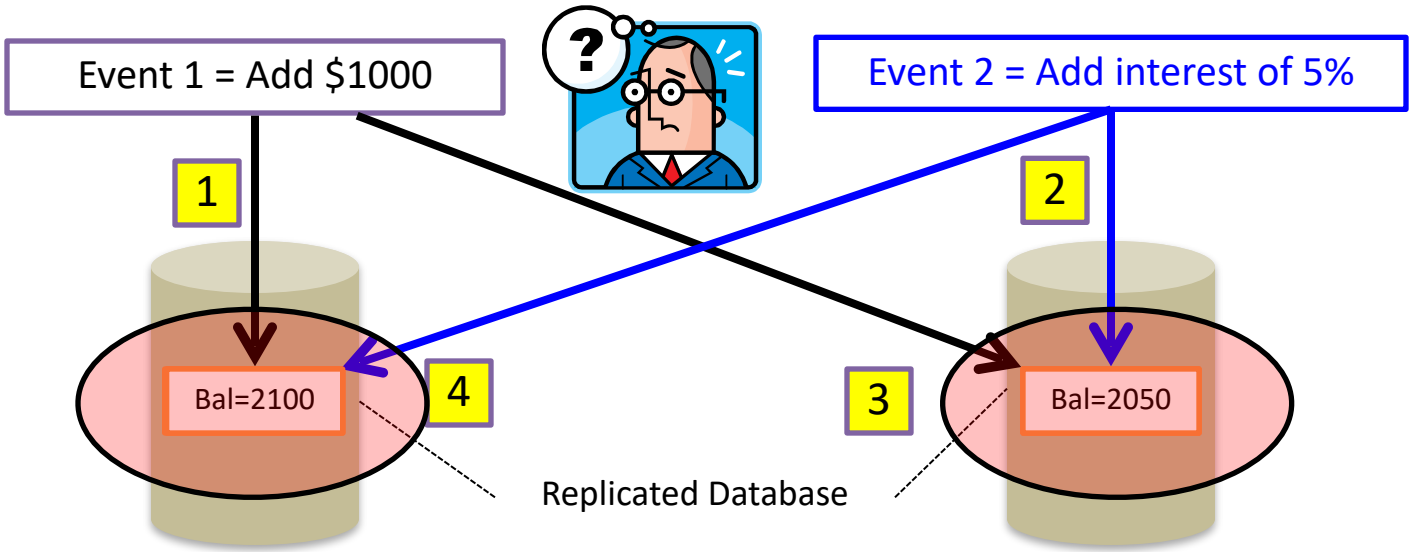
# Server System use

- Replicating data across servers helps in:
  - Avoiding performance bottlenecks
  - Avoiding single point of failures
  - And, hence, enhancing scalability and availability



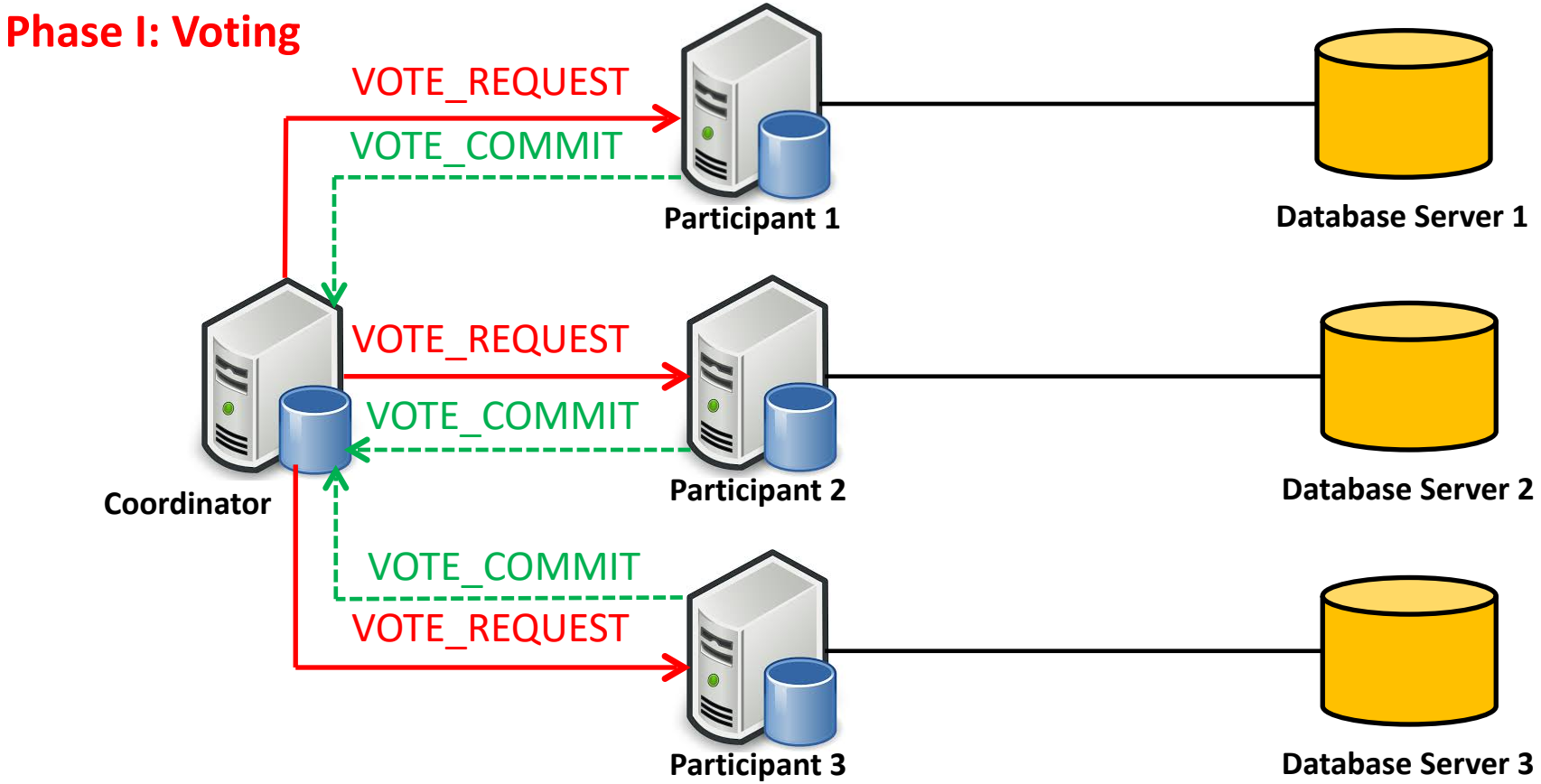
# But, Consistency Becomes a Challenge

- An example:
  - In an e-commerce application, the bank database has been replicated across two servers
  - Maintaining consistency of replicated data is a challenge



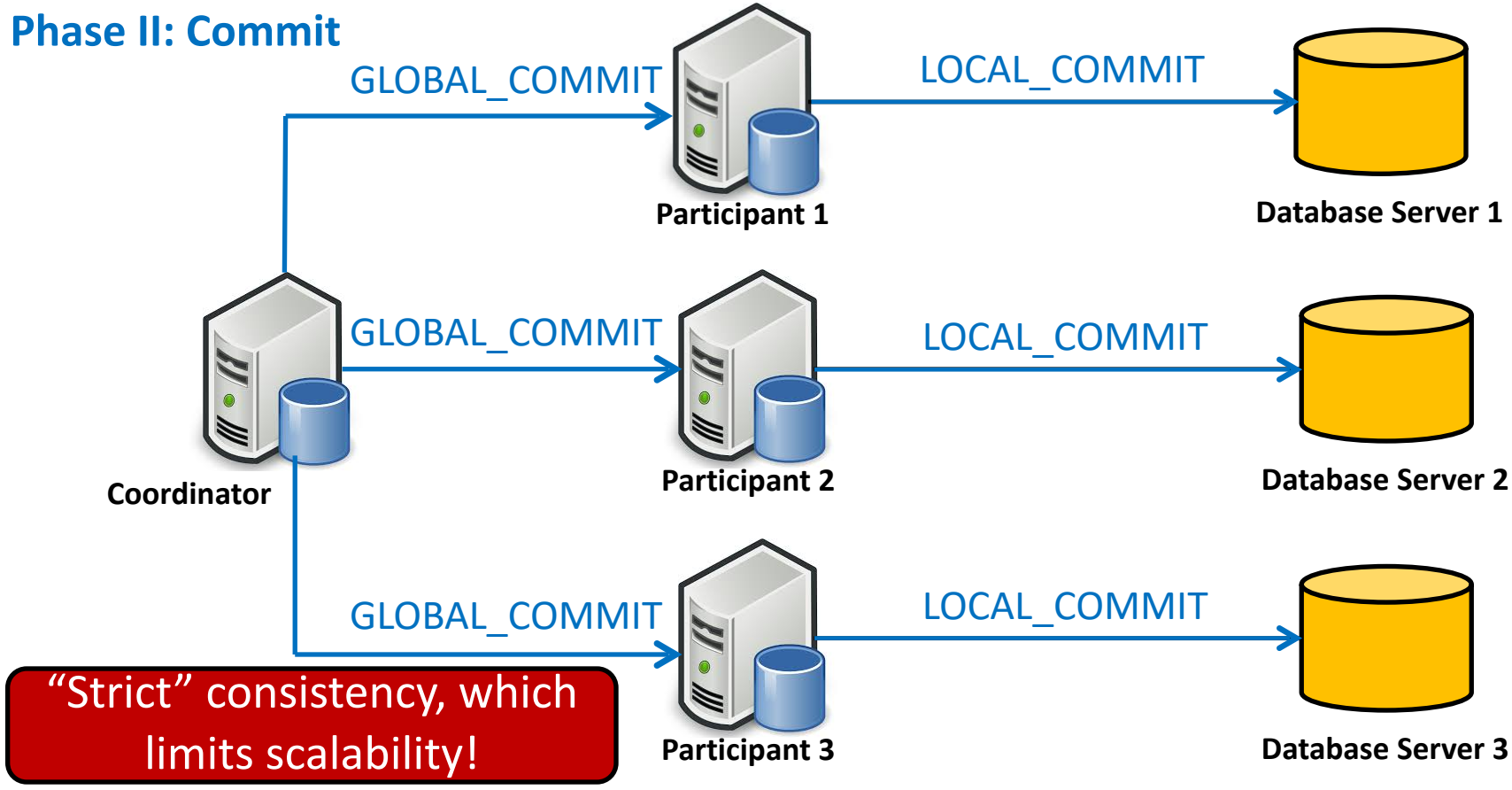
# The Two-Phase Commit Protocol

- The two-phase commit protocol (2PC) can be used to ensure atomicity and consistency



# The Two-Phase Commit Protocol

- The two-phase commit protocol (2PC) can be used to ensure atomicity and consistency



# Large-Scale Databases

- When companies such as Google and Amazon were designing large-scale databases, 24/7 Availability was a key
  - A few minutes of downtime means lost revenue

- When *horizontally* scaling databases to 1000s of machines, the likelihood of a node or a network failure increases tremendously

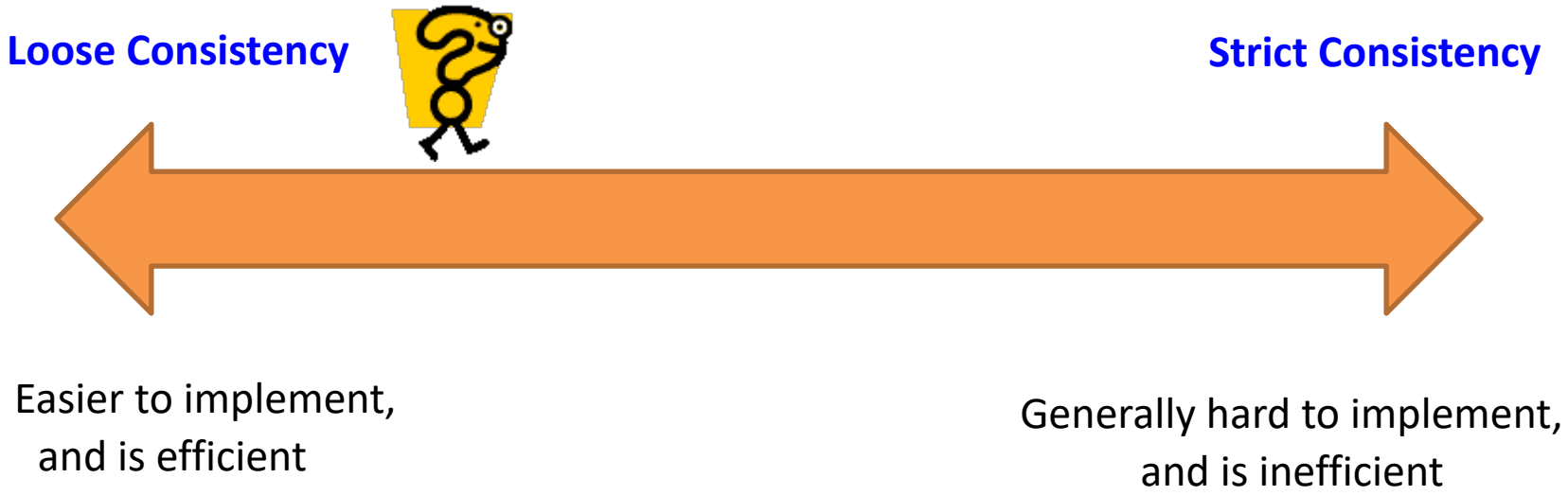
- Therefore, in order to have strong guarantees on Availability and Partition Tolerance, they had to sacrifice "strict" Consistency (*implied by the CAP theorem*)

# Trading-Off Consistency

- Maintaining consistency should balance between the strictness of consistency versus availability/scalability
  - Good-enough consistency *depends on your application*

# Trading-Off Consistency

- Maintaining consistency should balance between the strictness of consistency versus availability/scalability
  - Good-enough consistency *depends on your application*



# The BASE Properties

- The CAP theorem proves that it is impossible to guarantee strict Consistency and Availability while being able to tolerate network partitions

- This resulted in databases with relaxed ACID guarantees

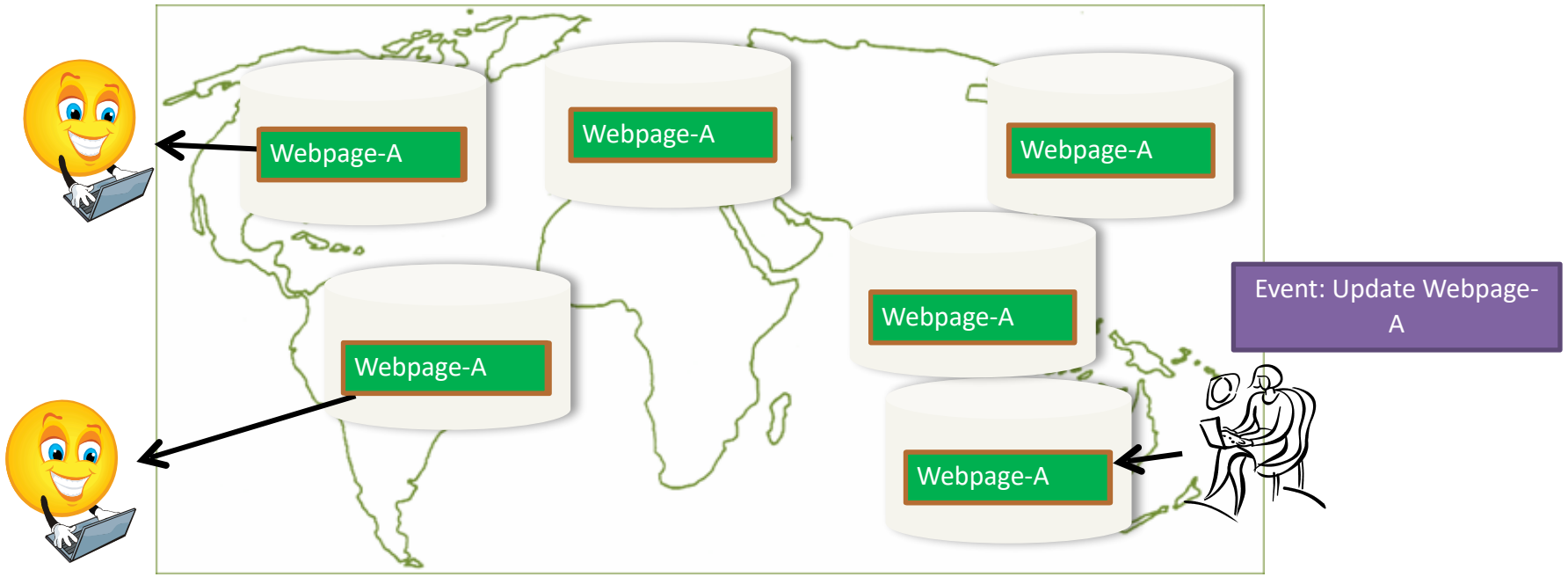
- In particular, such databases apply the BASE properties:
  - **B**asically **A**vailable: the system guarantees Availability
  - **S**oft-State: the state of the system may change over time
  - **E**ventual Consistency: the system will *eventually* become consistent

# Eventual Consistency

- A database is termed as *Eventually Consistent* if:
  - All replicas will *gradually* become consistent in the absence of updates

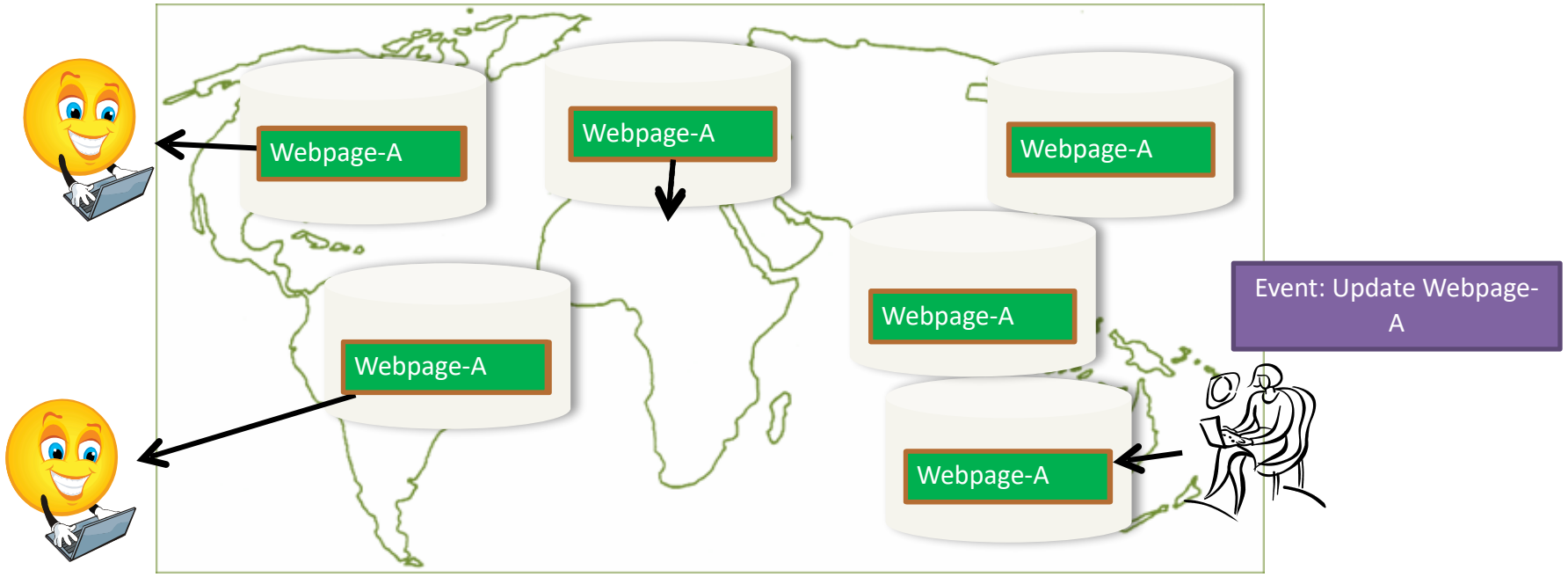
# Eventual Consistency

- A database is termed as *Eventually Consistent* if:
  - All replicas will *gradually* become consistent in the absence of updates



# Eventual Consistency: A Main Challenge

- But, what if the client accesses the data from different replicas?




Protocols like Read Your Own Writes (RYOW) can be applied!